Compte Rendu : TP API REST avec FastAPI (LogiAI)

Module : Développement et déploiement d'applications intelligentes | **Année :** 2026-2027 | **Groupe :** .....

Noms des étudiants : .....

1. Environnement et premiers tests

Sortie de python -m pytest au départ :

```
collected 13 items
tests\test_api.py ..FFFF......F                                          [100%]
============================ 5 failed, 8 passed in 4.07s ============================
```

Pourquoi certains tests attendant un code 422 passaient-ils déjà alors que rien n'était écrit ?

Dans le code starter, la classe Pydantic Livraison était configurée avec model_config = ConfigDict(extra="forbid") et ne contenait initialement que le champ distance_km. Dès qu'une requête de test envoyait les autres variables (poids_kg, heure_depart, etc.), Pydantic la rejetait immédiatement avec une erreur HTTP **422 Unprocessable Entity** pour champs inconnus interdits. Ainsi, les tests attendant un code 422 passaient par « faux vert » sans tester la validation métier des variables.

2. Schéma Pydantic (Livraison) et justification physique

Variable	Min	Max	Justification physique
distance_km	1	1000	Distance réelle en km d'une livraison routière (min 1 km, max environ 1000 km).
poids_kg	0.1	5000	D'un petit paquet léger (100 g) à la charge utile maximale d'un utilitaire / camionnette.
heure_depart	0	23	Heures valides d'une journée (horloge de 0h à 23h).
pluie_mm	0	100	Précipitations physiques positives ; 100 mm représente déjà un épisode pluvieux extrême.
trafic_index	0	10	Indice d'encombrement routier conventionnel : 0 = fluide, 10 = axe complètement saturé.
nb_arrets	0	30	Nombre d'arrêts intermédiaires sur la tournée (au moins 0, max réaliste 30 par tournée).
type_vehicule	-	-	Restreint à Literal["moto", "fourgon", "camion"] correspondant à la flotte disponible.

3. Implémentation de la fonction predict()

```
@app.post("/v1/predict", response_model=PredictionOut)
def predict(livraison: Livraison):
    predictor = state.get("predictor")
    if predictor is None:
        raise HTTPException(status_code=503, detail="Modèle non disponible / non chargé")
    return PredictionOut(**predictor.predict(livraison.model_dump()))
```

4. Réponses observées sur quatre requêtes types

Requête testée	Code	Explication technique
Livraison normale	**200**	Payload conforme au schéma Pydantic, le modèle est présent en mémoire et renvoie la prédiction.
distance_km = -5	**422**	Rejeté par Pydantic : violation de la borne minimale ge=1 sur distance_km.
type_vehicule = "avion"	**422**	Rejeté par Pydantic : la valeur ne fait pas partie du Literal["moto", "fourgon", "camion"].
Modèle absent (sans models/)	**503**	state.get("predictor") renvoie None, l'endpoint lève une HTTPException(503).

5. Questions de compréhension

1. Pourquoi charge-t-on le modèle au démarrage et pas à chaque requête ?

La lecture du fichier sur le disque (joblib.load) et la désérialisation prennent plusieurs centaines de millisecondes. En le chargeant au démarrage via lifespan, le modèle reste résident en RAM et chaque inférence s'exécute en quelques millisecondes sans surcharger le processeur.

2. Qui écrit le contrôle « pluie <= 100 », et où est-il écrit ?

C'est le développeur qui l'exprime de manière déclarative dans la classe Pydantic Livraison du fichier logiai/api/schemas.py, à l'aide de l'argument le=100 dans Field().

3. Quelle est la différence fondamentale entre les statuts 422 et 503 ?

Le code **422 (Unprocessable Entity)** signale une erreur côté **client** (requête JSON valide mais données hors bornes). Le code **503 (Service Unavailable)** signale un état côté **serveur** (le serveur HTTP tourne, mais la dépendance modèle ML n'est temporairement pas disponible).

4. Pourquoi utilise-t-on la route versionnée /v1/predict plutôt que simplement /predict ?

Le versionnement (/v1) permet de faire évoluer le contrat d'API (ex: une future /v2) sans casser le fonctionnement des clients existants (applications mobiles, partenaires, web).

5. **Que se passerait-il si la variable distance_km était déclarée de type int ?**

   FastAPI tenterait de forcer la conversion en entier ou rejetterait les nombres décimaux. Une distance réelle comme 12.5 km serait tronquée à 12 km ou refusée avec un code 422.

6. Décision de conception : comportement en cas de modèle absent

• **Option A (Démarrer et répondre 503) :** Permet de conserver l'API accessible aux outils de diagnostic et supervision. Le endpoint /health peut renvoyer {"status": "ok", "model_loaded": false}, ce qui permet à un opérateur de comprendre la cause de la panne. En revanche, le serveur reçoit du trafic client qu'il ne peut honorer.

• **Option B (Crash au démarrage / Fail Fast) :** Très adaptée aux orchestrateurs (Kubernetes/Docker). Si le modèle est absent, le conteneur ne passe jamais en état *Ready* ; aucun trafic client ne lui est envoyé et les équipes SRE sont immédiatement alertées.

7. Lien de la Pull Request & Tests

• **Dépôt GitHub :** https://github.com/malmo23/logiai | **Branche :** tp-api vers main
• **Lien Pull Request :** https://github.com/malmo23/logiai/pull/1
• **Validation pytest :** 15 passed in 3.62s (13 tests initiaux + 2 tests personnels ajoutés)